


# Programming and Algorithms

## **Aula 10:** Estruturas de repetição aninhadas

**Prof. Ettore Baldini-Neto**
ettore.neto@faculdadeimpacta.com.br

# Aulas Passadas

- Estrutura de repetição indefinida While
- Variáveis acumuladoras e contadoras
- Flag booleanas nos laços while
- Estruturas de repetição infinitas
- Alguns exercícios

# Programa da Aula

- Comandos de encerramento e interrupção de laços
- Estrutura de repetição repeat – until com while
- Loops de repetição para validação da entrada
- Estruturas de repetição aninhadas
- Alguns problemas com repetição aninhadas

# Motivações

- Há situações em que um laço de repetição precisa ser interrompido e o Python tem recursos para faze-lo.
- Pode-se simular um tipo de estrutura de repetição que não é nativa do Python mas que permite a resolução elegante de alguns problemas
  - Com essa nova simulação pode-se fazer validações de dados de entrada que são importantes em codificações que interagem com o usuário, por exemplo.

# O que já fizemos e onde estamos?

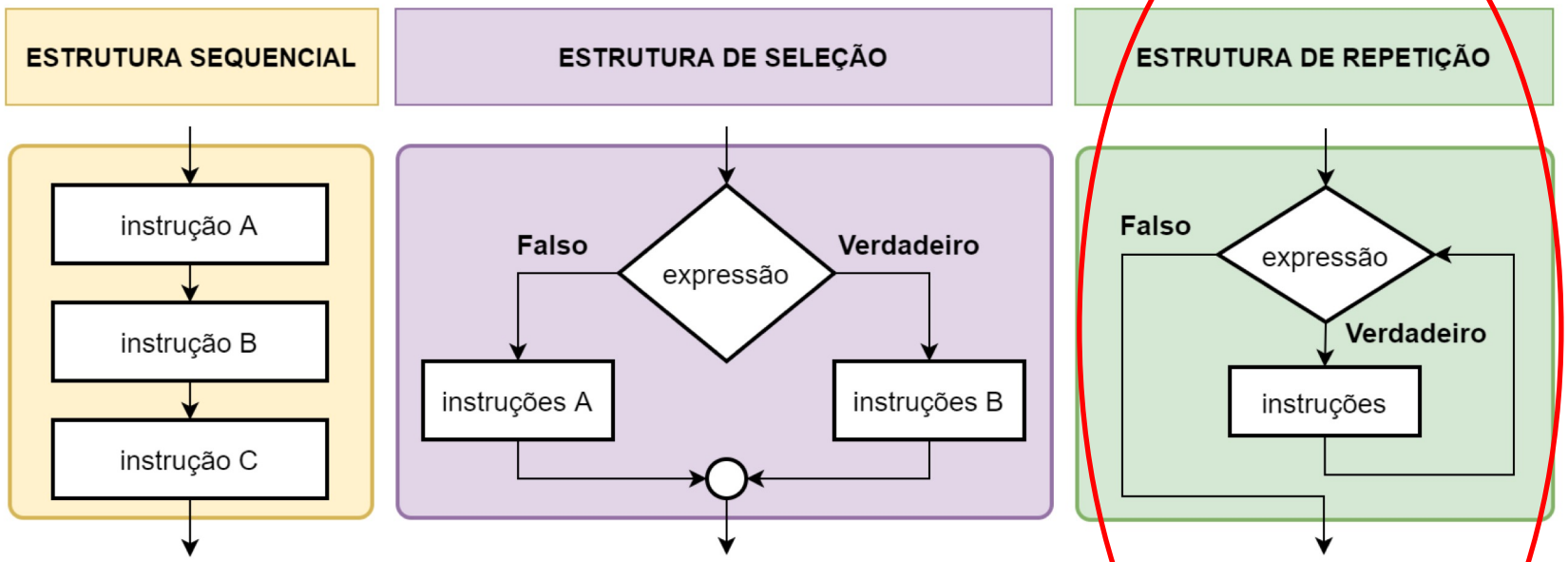
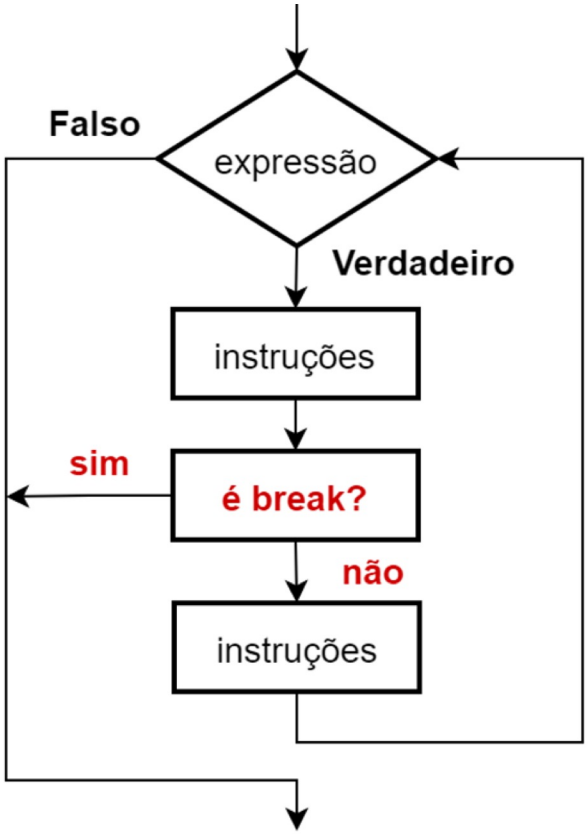



**Figura 7.1: Representação em fluxograma das estruturas básicas de controle de fluxo.**
**Fonte: Elaborado pelo autor.**

# Comando break

- Vamos nos aprofundar um pouco mais em recursos para a utilização de estruturas de repetição
- Embora seja útil, o comando while somente verifica sua condição de parada **no início** de cada repetição. Pode ser interessante, dependendo do problema, **terminar o while dentro do bloco.**
- O comando **break** é utilizado para **interromper** a execução do while **independentemente do valor atual de sua condição.**

# O comando break




Caso o comando **break** seja executado o laço é encerrado e o fluxo de execução será direcionado para a próxima instrução fora dele.

Portanto, é comum que o comando **break** esteja dentro do bloco de instruções de uma estrutura de seleção, de modo que só seja executado se uma condição específica for satisfeita.

# O comando break

- A execução do comando break encerra o loop **mais interno** em que está contido.
- Geralmente o break está condicionado à um comando **if,** de tal modo que somente seja executado quando algo específico ocorrer

# O comando break

**Exemplo:** Crie um programa que receba como entrada o crédito de um cliente e depois o preço de itens comprados por esse cliente. O programa deve parar de solicitar novos preços quando o crédito disponível for insuficiente para pagar um dos itens. Ao final, o programa deve exibir o crédito restante.

# O comando break

```
credito = float(input('Seu crédito : R$ '))
while credito > 0:
    item = float(input('preço do item: R$ '))
    if item > credito:

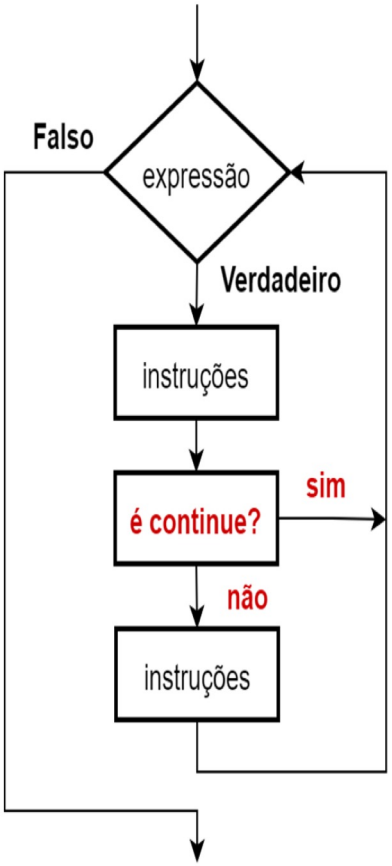
        print('Compra negada! Ultrapassa seu crédito.')
        break
    credito -= item
print(f'Crédito restante : R$ {credito:.2f}')
```

```
Seu crédito : R$ 100.00
preço do item: R$ 34.00
preço do item: R$ 23.00
preço do item: R$ 21.00
preço do item: R$ 14.00
preço do item: R$ 34.00
Compra negada! Ultrapassa seu crédito.
Crédito restante : R$ 8.00
```

# O comando continue

- A execução de um comando continue **interrompe a rodada atual do loop mais interno em que está contido.**
- Geralmente está condicionado a um **if** de tal modo que somente seja executado se algo específico ocorrer.

# O comando continue




Caso o comando **`continue`** seja executado a rodada atual do laço é interrompida e o fluxo de execução será direcionado para a condição do *loop*.

Portanto, é comum que o comando **`continue`** esteja dentro do bloco de instruções de uma estrutura de seleção, de modo que só seja executado se uma condição específica for satisfeita.

**Figura 8.2: Representação simplificada do comando `continue` em um fluxograma.**
**Fonte: Elaborado pelo autor.**

# O comando continue

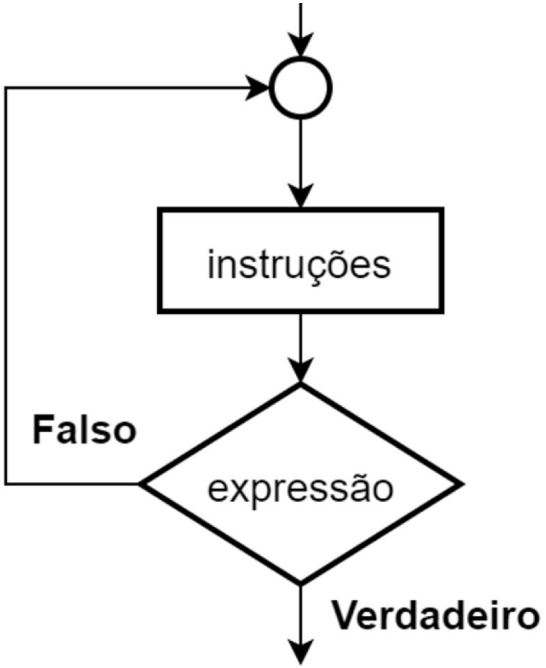
```
credito = float(input('Seu crédito é de: R$ '))
while credito > 0:
  item = float(input('Preço do item: R$ '))
  if item > credito:
    print(f'Compra negada! Restam: R$ {credito:.2f}')
    continue
  credito -= item
print(f'Você zerou sua carteira!')
```

```
Seu crédito é de: R$ 100.00
Preço do item: R$ 25.00
Preço do item: R$ 25.00
Preço do item: R$ 60.00
Compra negada! Restam: R$ 50.00
Preço do item: R$ 25.00
Preço do item: R$ 45.00
Compra negada! Restam: R$ 25.00
Preço do item: R$ 25.00
Você zerou sua carteira!
```

# while ...True -Simulação de repeat ...until-

- Essa estrutura obriga que o bloco de instruções do loop seja executado **pelo menos uma vez.**
- Diferentemente do while, a condição para repetição está no **fim do loop** e não no início.
- Então, para que o fluxo de execução atinja a condição deste loop é necessário que primeiro passe por seu bloco de código.

# Simulação de repeat ...until com while...True




O laço **repeat…until** possui uma demarcação inicial, para indicar seu início, seguida por um bloco de instruções e, por fim, sua expressão de decisão, ou seja, a condição do *loop*, que também indica seu fim.

Após a primeira execução do bloco de instruções, a condição é avaliada, caso resulte em **falso** o fluxo de execução é direcionado para a demarcação inicial, caso resulte em **verdadeiro**, o laço é encerrado.

**Figura 8.3: Representação do laço *repeat…until*. Fonte: Elaborado pelo autor.**

**sintaxe:** while True:

<bloco de instruções>

if < condição >: break

# repeat ...until

```
s = 0
while True:
    v = int(input('Digite um número para somar ou 0 para sair: '))
    if v == 0:
        break
    s += v
print(s)
```

Veja o fluxo de execução no python tutor:

Note que a condição **True** faz o while ser executado **indefinidamente.**

O break **está dentro do bloco do if,** especificamente quando v = 0**.**

- Enquanto v for diferente de zero, a repetição continuará a somar v à s**.**
- Quando v = 0, o break será executado, terminando a repetição e finalmente imprimindo o valor de s

Python 3.6

known limitations

```
1 s = 0
2 while True:
3     v = int(input('Digite um numero para somar ou
4     if v == 0: break
5     s += v
6 print(s)
```

Edit this code

line that just executed

next line to execute

<< First | < Prev | Next > | Last >>

Done running (14 steps)

Print output (drag lower right corner to resize)

```
Digite um numero para somar ou 0 para sair: 1
Digite um numero para somar ou 0 para sair: 2
Digite um numero para somar ou 0 para sair: 3
Digite um numero para somar ou 0 para sair: 0
6
```

Frames Objects

Global frame

| | |
|---|---|
| s | 6 |
| v | 0 |

# while ...True

Execute no python tutor

```
[3]  def invertido(n):
         while True:
             print(n % 10, end = '')
             n = n // 10
             if n == 0:
                 break
```

```
invertido(123)
```

# Validação de dados de entrada

Normalmente a validação de dados de entrada é feita com laços, solicitando um valor para o usuário repetidas vezes até que seja inserido algum dado válido que é esperado pelo algoritmo.

**Exemplo:** Crie um programa que receba como entrada um número natural n no intervalo [0,100] e exiba a soma dos n naturais positivos. A soma deverá ser feita por uma função, que a devolverá como resposta. O programa deverá validar a entrada e só chamará a função quando n for válido.

# Validação de dados de entrada

```
def somatorio(n):
    soma = 0
    natural = 1
    while natural <= n:
        soma += natural
        natural += 1
    return soma

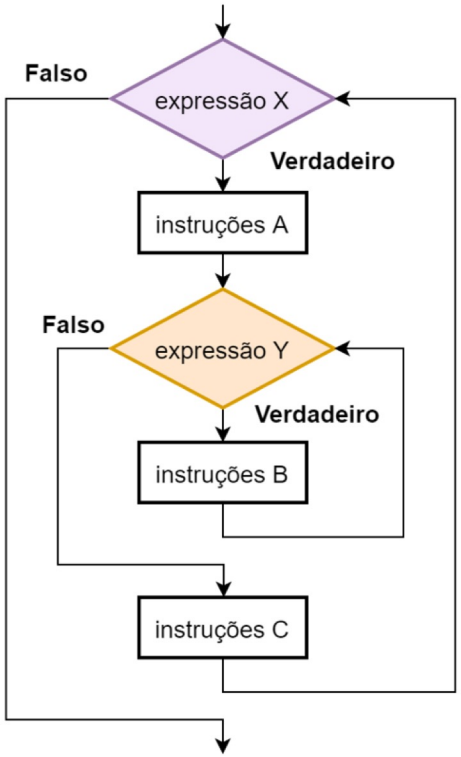
while True:
    n = int(input('Entre com um natural entre 1 e 100: '))
    if 0 <= n <= 100:
        break

print(f'A soma dos {n} primeiros naturais é {somatorio(n)}')
```

A chamada à função somatório só vai ocorrer caso o fluxo de execução passe pela validação da entrada feita com o laço while. O programa impede um erro caso sejam inseridos valores menores que zero ou maiores que 100. O resultado deve concordar com $S = \frac{n(n+1)}{2}$

# Estruturas de repetição aninhadas

São basicamente loops dentro de blocos de códigos de outros loops.




A cada rodada do *loop* mais externo (expressão X) o *loop* mais interno (expressão Y) executa todas suas rodadas. Dizemos que o *loop* mais interno está aninhado em relação ao mais externo.

```
...
while expressão X:
    instruções A
    while expressão Y:
        instruções B
    instruções C
...
```

**Figura 8.4: Laço aninhado em fluxograma e em Python. Fonte: Elaborado pelo autor.**

# Estruturas de repetição aninhadas

**Exemplo:** Criando um relógio.
Crie uma função que exibe todos os segundos de um minuto.

```
def relogio():
  s = 0
  while s < 60:
    print(s)
    s += 1
# vai imprimir os segundos de 1 a 59 sem qualquer formatação
```

```
from time import sleep  # chamamos o modulo sleep para mudar a exibição
def relogio():
  s = 0
  while s < 60:
    print(f'00:00:{s:02}')
    sleep(1)
    s += 1
```

# Estruturas de repetição aninhadas

A cada 60 segundos o contador dos segundos zera e o de minutos aumenta.

```python
# inserindo o laço dos minutos e alterando a exibição para mostrar minutos tambem
from time import sleep
def relogio():
  m = 0
  while m < 60:
    s = 0
    while s < 60:
      print(f'00:{m:02}:{s:02}')
      sleep(1)
      s += 1
    m += 1
```

# Estruturas de repetição aninhadas

Incluindo o laço das horas e exibindo o relógio completo.

```
#inserindo o laço das horas e alterando a exibição para mostrar tudo
from time import sleep
def relogio():
  h = 0
  while h < 24:
    m = 0
    while m < 60:
      s = 0
      while s < 60:
        print(f'{h:02}:{m:02}:{s:02}')
        sleep(0.01)
        s += 1
      m += 1
  h += 1
```

Note que o parâmetro de sleep foi mudado para o relógio andar mais rapido.

Por padrão use sleep(1)

# Estrutura de repetição aninhada

Criando um relógio: Altere a função para que a cada 24h os três mostradores sejam zerados e o relógio reinicie a contagem: Mais um loop que desta vez contenha todos os anteriores. Usaremos um laço infinito para que o relógio nunca pare.

+ Code + Text

```
[12] from time import sleep
     def relogio():
       while True:
         h = 0
         while h < 24:
           m = 0
           while m < 60:
             s = 0
             while s < 60:
               print(f'{h:02}:{m:02}:{s:02}')
               sleep(1)
               s += 1
             m += 1
           h += 1
```

# Estrutura de repetição aninhada.

**Exemplo:** Crie uma função que receba como parâmetro um número natural n e desenhe um tabuleiro n x n no qual os quadrados escuros seja desenhados com dois caracteres Unicode 9608 e os claros com dois espaços.

# Estrutura de repetição aninhada

```
def tabuleiro(n):
    linha = 0
    while linha < n:
        coluna = 0
        while coluna < n:
            if (linha + coluna) % 2 == 0:
                print( 2 * chr(9608), end = '')
            else:
                print(2 * ' ', end = '')
            coluna += 1
        print()
        linha += 1
```

```
tabuleiro(5)
```



# Exercícios

## VAMOS PRATICAR!

2) Crie uma nova versão da função `relogio` em que o horário inicial seja 23:59:59 e a função exiba a contagem regressiva até 00:00:00.

3) Refaça a Codificação 10.14 de modo que o tabuleiro não seja necessariamente quadrado, ou seja, o número de linhas possa ser diferente do número de colunas. Note que será necessário incluir mais um parâmetro na função para representar as colunas.

4) Crie um programa que chame uma função com três parâmetros. O primeiro parâmetro é um natural `linhas`, o segundo é um natural `colunas` e o terceiro um caractere `s`. A função deve exibir um retângulo de tamanho `linhas` x `colunas` composto apenas por `s`, conforme exemplo à direita.



# FIM